Molecular Dynamics Simulation of a Lennard-Jones Liquid

Vedant Neema

April 29, 2026

Description

The program [1] in this repository simulates a Lennard-Jones Liquid with variables: $N = 100$ particles in a 2D square box with side, $L = 11$ ($\Rightarrow \rho = 0.83$), with velocities distributed as per $T = 1$

Dependencies

I use the **Raylib** Graphics Library v5.5-1 for the interface and **gnuplot** v6.0 for the graphs.

How to run

Look for the file `ljliquid.c`, get Raylib 5.0 or higher, then compile using the command

```
$> gcc ljliquid.c -lm -lraylib -o ljliquid
```

This should produce a binary that can be run in the terminal like

```
$> ./ljliquid
```

Plots

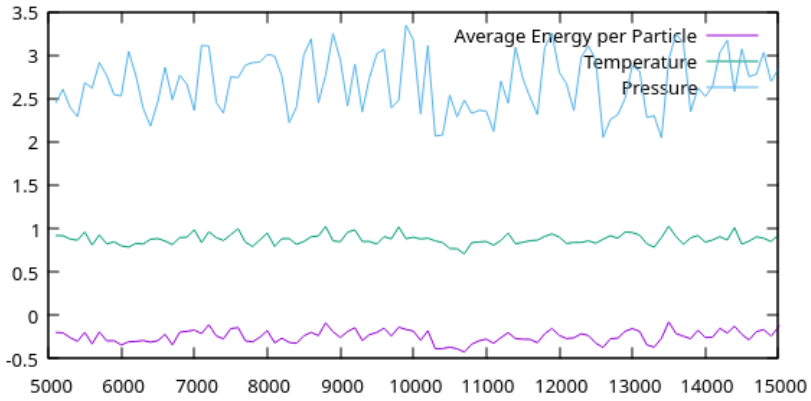

Figure 1: Graph of energy, temperature and pressure

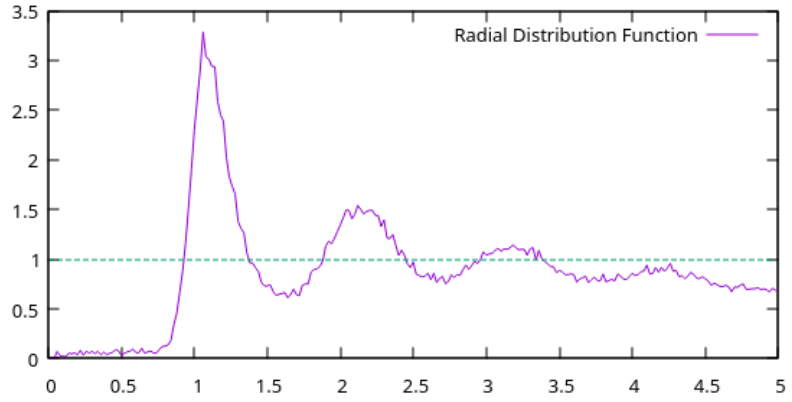

Figure 2: Radial Distribution Function (rdf) sampled during the production run

Principles used

Using the velocity Verlet algorithm, we get a order Δt^4 accuracy in position, Δt^3 in velocity and Δt^2 in Energy[2]:

- Calculate $\mathbf{x}(t+\Delta t) = \mathbf{x}(t) + \mathbf{v}(t)\,\Delta t + \frac{1}{2}\,\mathbf{a}(t)\,\Delta t^2$.
- Derive $\mathbf{a}(t+\Delta t)$ from the interaction potential using $\mathbf{x}(t+\Delta t)$.
- Calculate $\mathbf{v}(t+\Delta t) = \mathbf{v}(t) + \frac{1}{2}\,\big(\mathbf{a}(t) + \mathbf{a}(t+\Delta t)\big)\Delta t$.

Observations

The rdf of the system looks like that of liquids. So, we can assume under the given conditions (T = 1, ρ = 8.3), any particles with the Lennard-Jones potential will be in the liquid state.

Code

```c
#include <math.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <raylib.h>
#include <raymath.h>

/* * Constants and Macros */
#define N 100
#define L 11
#define M 500

#define MENU 0
#define SW (M+MENU)
#define SH (M>100?M:100)

#define A(v, w) Vector2Add(v, w)
#define S(v, w) Vector2Subtract(v, w)
#define C(v, s) Vector2Scale(v, s)
#define R(v) Vector2Length(v)

/* * Structs and Functions */
typedef struct particle {
  Vector2 qq, q;
  Vector2 p, a;
  Color c;
} particle;

const double delta = 0.005;
double K = 0, U = 0, P = 0, p = 0;
particle box[N];
bool production = false;
#define BIN_COUNT 250
```

```
double rdf_bins[BIN_COUNT] = {};
double lj_potential(double r) {return  4 * (1/(1+powf(r, 12)) - 1/(1+powf(r, 6)));}
double     lj_force(double r) {return 24 * (2/(1+powf(r, 13)) - 1/(1+powf(r, 7)));}

void initialize_particles() {
  for (int i = 0; i < BIN_COUNT; i++) {
    rdf_bins[i] = 0.;
  }
  for (int i = 0; i < N; i++) {
    box[i].q = (Vector2) {random()%L, random()%L};
    double _theta = random()%360 * (2*PI/360);
    double _r = random()%101 / 10.;
    box[i].p = (Vector2) {_r * cos(_theta), _r * sin(_theta)};
    box[i].qq = S(box[i].q, C(box[i].p, delta));
    box[i].c = (Color) {random()%255, random()%255, random()%255, 255};
  }
  /* Normalize Temperature = 1 */
  double ptotal = 0;
  for (int i = 0; i < N; i++) {
    ptotal += Vector2LengthSqr(box[i].p);
  }
  for (int i = 0; i < N; i++) {
    box[i].p = C(box[i].p, 2*N/ptotal);
  }
}

void log_stats(int iteration) {
  K = 0;
  for (int i = 0; i < N; i++) {
    K += 0.5 * Vector2LengthSqr(box[i].p);
  }
  double T = K/N;
  U = 0;
  p = T * N/L/L;
  for (int i = 0; i < N; i++) {
    for (int j = 0; j < i; j++) {
      double r = Vector2Distance(box[i].q, box[j].q);
      U += lj_potential(r);
      p += lj_force(r) * r/2/L/L;
      if (production) if (r < 5) rdf_bins[(int) (r*50)]++;
    }
  }
  printf("%d, %f, %f, %f\n", iteration, (K+U)/N, T, p);
}

/* * Main */
int main()
{
  bool paused = false;
  InitWindow(SW, SH, "Lennard-Jones Liquid");
  srandom(42);
  SetTargetFPS(60);
  ToggleFullscreen();
  ToggleFullscreen();
```

```
88
89  bool constraining = true;
90  double time_collector = 0;
91  int timecent = 0;
92  initialize_particles();
93
94  while (!WindowShouldClose())
95  {
96    time_collector += GetFrameTime();
97    if (IsKeyPressed(KEY_SPACE)) { paused = !paused; time_collector
    =0; }
98    if (IsKeyPressed(KEY_C)) constraining = !constraining;
99    if (paused) goto draw;
100
101   /* Update */
102   while (time_collector > delta) {
103     timecent++;
104     if (timecent >= 5000) production = true;
105     if (timecent%100 == 0)
106       log_stats(timecent);
107     if (timecent >= 15000) paused = true;
108     time_collector -= delta/7;
109     for (int i = 0; i < N; i++) {
110       box[i].q = A(A(box[i].q, C(box[i].p, delta)), C(box[i].a,
    delta*delta/2));
111       Vector2 aa = Vector2Zero();
112       for (int j = 0; j < N; j++) {
113         if (i==j) continue;
114         double r = Vector2Distance(box[i].q, box[j].q);
115         if (r == 0) continue;
116         aa = A(aa, C(S(box[j].q, box[i].q), -lj_force(r)/r));
117       }
118       box[i].p = A(box[i].p, C(A(box[i].a, aa), delta/2));
119       double r = R(box[i].p);
120       if (r > 2)
121         box[i].p = C(box[i].p, 2./r);
122       box[i].a = aa;
123     }
124   }
125
126   /* PBC */
127   for (int i = 0; i < N; i++) {
128     if (box[i].q.x < 0) box[i].q.x += L;
129     if (box[i].q.y < 0) box[i].q.y += L;
130     if (box[i].q.x > L) box[i].q.x -= L;
131     if (box[i].q.y > L) box[i].q.y -= L;
132   }
133 draw:
134   /* Draw: box */
135   BeginDrawing();
136   ClearBackground(BLACK);
137   for (int i = 0; i < N; i++) {
138     /* DrawPixelV(C(box[i].q, M/L), box[i].c); */
139     DrawCircleV(C(box[i].q, M/L), 2, box[i].c);
140   }
141
142   /* Draw: menu */
```

```
143     DrawRectangle(M, 0, MENU, SH, DARKGRAY);
144     char buf[1024];
145     int height = 1;
146     DrawFPS(M+1, 0);
147     snprintf(buf, 1024, "Kinetic Energy: %f", K);
148     DrawText(buf, M+1, 16*height++, 16, WHITE);
149     snprintf(buf, 1024, "Potential Energy: %f", U);
150     DrawText(buf, M+1, 16*height++, 16, WHITE);
151     snprintf(buf, 1024, "Total Energy: %f", K+U);
152     DrawText(buf, M+1, 16*height++, 16, WHITE);
153     if (production) {
154       snprintf(buf, 1024, "Production: true");
155       DrawText(buf, M+1, 16*height++, 16, WHITE);
156     }
157 
158     EndDrawing();
159   }
160   for (int i = 0; i < BIN_COUNT; i++) {
161     printf("%f, %f, 1\n", i/50., rdf_bins[i]/powf(i+1, 0.95)/70);
162   }
163   CloseWindow();
164 
165   return 0;
166 }
```

Ending

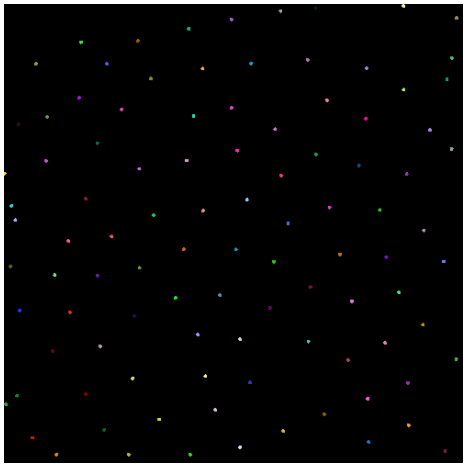

Figure 3:

References

[1] Repository containing ljliquid.c on Github

[2] Velocity Verlet on Wikipedia